

1h30 - Sans documents excepté une copie double resto-verso avec notes manuscrites

La clarté de vos réponses sera prise en compte. Ne pas écrire au crayon.

N'oubliez pas de commenter vos programmes

Vous aurez besoin de 2 copies : une par exercice

Attention à bien conserver l'indentation tout au long de votre copie. Vous pouvez écrire en commentaire dans votre programme ou dans la marge le n° de la question. Lorsque les setter et getter ne sont pas demandés, il n'est pas nécessaire de les écrire.

1. Covoiturage (10 points) -> copie n°1

On souhaite créer une plateforme de covoiturage pour les étudiants et personnels de l'UTC. Votre programme devra permettre aux utilisateurs d'ajouter des trajets et de réserver des places dans les covoiturations.

Un objet de la classe *Covoit* est composé de :

- ✓ un conducteur : login du conducteur
- ✓ une source : nom de la ville de départ
- ✓ une destination : nom de la ville arrivée
- ✓ une date_depart de type datetime
- ✓ un nombre de places *disponibles* nb_places sous la forme d'un entier
- ✓ une liste de covoitureurs passagers : la liste des logins des passagers, vide au départ

1.1 - Écrire la classe *Covoit* avec son constructeur ainsi que les getter pour la source, la destination et la date de départ. On ne fera aucune vérification sur les données entrées.

On souhaite désormais inscrire des utilisateurs dans un covoiturage. Quelques contraintes s'appliquent :

- ✓ le conducteur ne peut pas être passager
- ✓ un passager est unique
- ✓ le nombre de passagers ne peut pas dépasser nb_places

Si une des contraintes n'est pas remplie, on affichera un message d'erreur.

1.2 - Ajouter à la classe *Covoit* une méthode *ajouter_un_passager()* qui prend en paramètre un login et l'ajoutera comme passager au covoiturage.

1.3 - Ajouter à la classe *Covoit* une méthode *supprimer_un_passager()* qui supprimera le passager dont le login est passé en paramètre du covoiturage. S'il n'est pas passager on affichera une erreur.

On souhaite pouvoir comparer deux covoiturations proposés sur la plateforme afin d'éviter les doublons. Deux covoiturations sont égaux si :

- ✓ le conducteur est le même
- ✓ les sources et destinations sont les mêmes
- ✓ la date de départ est la même (d1 == d2 renvoie True si les dates sont égales)

1.4 - Ajouter à la classe *Covoit* la méthode *__eq__(self, other)* qui permet de tester si deux covoiturations sont égaux.

Dans le programme principal nous allons gérer une variable *covoits* qui contiendra la liste de tous les covoiturations.

1.5 - Écrire une fonction *ajouter_un_covoiturage()* qui prendra en paramètre un objet de type *Covoit* et une liste de covoiturations. Cette fonction ajoutera le covoiturage passé en paramètre à la liste s'il n'est pas déjà présent.

1.6 - Écrire une fonction *rechercher_covoiturage()* qui prendra en paramètres la liste des covoiturations et trois critères : la date de départ de type *datetime*, la ville de départ, la ville d'arrivée. La fonction devra retourner la **liste** des covoiturations correspondant aux critères de recherche.

Un covoiturage correspond aux critères quand :

- ✓ les villes de départ et d'arrivée sont les mêmes
- ✓ le décalage entre la date de départ du covoiturage et la date recherchée par l'utilisateur est inférieur à 1h (utilisez la fonction *calcul_decalage* définie ci-dessous).

On dispose d'une fonction *calcul_decalage(date1, date2)* qui prend deux dates en paramètres et renvoie le nombre de minutes séparant les deux dates (cette fonction n'est pas à écrire).

Exemple de corrigé :

```
from datetime import datetime
from typing import List

class Covoit:
    # Aucune vérification -> pas d'intérêt à encapsuler
    def __init__(self, conducteur, source, destination, date_depart, nb_places):
        self.conducteur = conducteur
        self.source = source
        self.destination = destination
        self.date_depart = date_depart
        self.nb_places = nb_places
        self.passagers = [] # vide au départ

    def ajouter_un_passager(self, passager):
        # ATTENTION -> Afficher les erreurs pas lever des exceptions
        if passager == self.conducteur:
            print("Le passager ne peut pas être conducteur")
            return

        if passager in self.passagers:
            print("Le passager est déjà passager")
            return

        if len(self.passagers) >= self.nb_places:
            print("Covoit plein")
            return
        self.passagers.append(passager)

    def supprimer_un_passager(self, passager):
        if passager not in self.passagers:
            print("Cette personne n'est pas passager")
            return
        self.passagers.remove(passager)

    def __eq__(self, other):
        return self.conducteur == other.conducteur and \
            self.source == other.source and self.destination == other.destination and \
            self.date_depart == other.date_depart

# Variable globale, "typage" pas obligatoire
covoits: List[Covoit] = []

def creer_covoiturage(c):
    if c not in covoits:
        covoits.append(c)

# supposée connue
def calcul_decalage(d1, d2):
    pass

def rechercher_covoiturage(ville_depart, ville_arrivee, date_souhaitee):
    resultats = []
    for c in covoits:
        if c.source == ville_depart and c.destination == ville_arrivee and \
            calcul_decalage(c.date_depart, date_souhaitee) <= 60:
            resultats.append(c)
    return c

def main_ex1():
    c1 = Covoit("Loïc", "Compiegne", "Rennes", datetime(2022, 4, 18, 9, 40), 4)
    c1.ajouter_un_passager("Manon")
    c1.ajouter_un_passager("Jean")
    c1.ajouter_un_passager("Maxime")
    c1.supprimer_un_passager("Maxime")
```

```

c1.supprimer_un_passager("non existant") # affiche Erreur
c1.ajouter_un_passager("Lola")
c2 = Covoit("Loïc", "Rennes", "Compiègne", datetime(2022, 4, 18, 10, 40), 4)
c3 = Covoit("Thomas", "Rennes", "Compiègne", datetime(2022, 4, 18, 9, 40), 4)

print(c3 == c2) # True
creer_covoiturage(c1)
creer_covoiturage(c2)
creer_covoiturage(c3)
print(f"Correspondant = {rechercher_covoiturage('Rennes', 'Compiegne', datetime(2022, 4,
18, 9, 40))}")

```

2. Stellar View

```

class ObjetCeleste:
    __designation_objet_dict = {} # question 2.4 V2

    def __init__(self, designation):
        self.__designations = [] # attribut encapsulé manipulé par "ajouter_designation" et
"get_designations"
        self.ajouter_designation(designation)

    def ajouter_designation(self, designation):
        if not isinstance(designation, str):
            raise TypeError("La désignation doit être de type str")
        if designation in self.__designations:
            raise ValueError("Désignation déjà existante")
        self.__designations.append(designation)
        ObjetCeleste.__designation_objet_dict[designation] = self # question 2.4 V2

    def __str__(self):
        return f"designations = {self.__designations}"

    def get_designations(self):
        return self.__designations

    @staticmethod # question 2.4 V2
    def get_objet_celeste(designation):
        return ObjetCeleste.__designation_objet_dict.get(designation, None)

# Etoile et Constellation sont des ObjetCeleste
# Etoile avec getter et setter manuel
class Etoile(ObjetCeleste):
    def __init__(self, designation, magnitude):
        super().__init__(designation)
        self.set_magnitude(magnitude)

    def get_magnitude(self):
        return self.__magnitude

    def set_magnitude(self, magnitude):
        if not isinstance(magnitude, float):
            raise TypeError("Magnitude doit être un float")
        self.__magnitude = magnitude

    def __str__(self):
        return
f"Etoile\n\tdesignations={self.get_designations()}\n\tmagnitude={self.get_magnitude()}"

# Etoile avec property
class EtoileProperty(ObjetCeleste):
    def __init__(self, designation, magnitude):
        super().__init__(designation)
        self.magnitude = magnitude

    @property
    def magnitude(self):
        return self.__magnitude

```

```

@magnitude.setter
def magnitude(self, magnitude):
    if not isinstance(magnitude, float):
        raise TypeError("Magnitude doit être un float")
    self.__magnitude = magnitude

class Constellation(ObjetCeleste):
    def __init__(self, designation, etoile_1, etoile_2):
        super().__init__(designation)
        self.__etoiles = [] # Aucune au début
        self.ajouter_etoile(etoile_1)
        self.ajouter_etoile(etoile_2)

    def ajouter_etoile(self, etoile):
        if not isinstance(etoile, Etoile):
            raise TypeError("L'étoile doit être de type Étoile")
        if etoile in self.__etoiles:
            raise ValueError("Étoile déjà présente dans la constellation")
        self.__etoiles.append(etoile)

    def __str__(self):
        return
        f"Constellation\n\tdesignations={self.get_designations()}\n\teetoiles=[e.get_designations()
        for e in self.__etoiles]"

## 2.4 Version 1
class Index:
    def __init__(self):
        self.__designation_objet_dict = dict()

    def indexer(self, objet_celeste: ObjetCeleste):
        for designation in objet_celeste.get_designations():
            self.__designation_objet_dict[designation] = objet_celeste

    def get_objet_celeste(self, designation):
        return self.__designation_objet_dict.get(designation, None)

def main_ex2():
    e_1 = Etoile("Etoile1", 1.4)
    e_1.ajouter_designation("Etoile1_1")
    e_2 = Etoile("Etoile2", 1.4)
    c = Constellation("Constellation 1", e_1, e_2)
    # get_objet_celeste avec Index
    index = Index()
    index.indexer(e_1)
    index.indexer(e_2)
    index.indexer(c)
    print(index.get_objet_celeste("Etoile1_1"))

    # get_objet_celeste avec ObjetCeleste
    print(ObjetCeleste.get_objet_celeste("Constellation 1"))

if __name__ == "__main__":
    main_ex1()
    main_ex2()

```

2. Stellar View (10 points) -> copie n°2

Le but de cet exercice est de développer des composants destinés à Stellar View, un logiciel de visualisation de cartes astronomiques. Une étoile est un corps céleste qui émet sa propre lumière. Une constellation est un ensemble d'étoiles (au moins 2) auquel on a donné un nom (ces étoiles projetées sur la voûte céleste forment une figure particulière). Une étoile ou une constellation peut posséder différents noms selon le système utilisé où la civilisation qui les ont imaginés. Par exemple les noms Aldébaran, Alpha-Tau, HR 1457 ou HD 29139 sont autant de noms qui désignent la même étoile. De même, les noms la Grande Ourse, Septem Triones, the Plough ou Karrkamm sont des noms qui désignent la même constellation. De manière générale, on utilise le concept d'objet céleste pour parler indifféremment d'une étoile ou d'une constellation.

Dans l'application, on représente les étoiles par des objets de la classe *Etoile* et les constellations par des objets de la classe *Constellation*. Ces deux classes sont des spécialisations de la classe *ObjetCeleste*, étoiles et constellations sont des *ObjetCeleste*.

Lorsque l'on exécute la fonction *print()* sur un objet de ces classes, toutes les caractéristiques (spécifiées ci-dessous) doivent être affichées, ce qui nécessite l'écriture des méthodes *__str__()* de chaque classe. Dans toutes les méthodes décrites ci-dessous, une exception doit être levée si les types des arguments fournis par l'utilisateur ne sont pas conformes à ceux attendus.

La classe *ObjetCeleste* possède un attribut qui permet de stocker une liste de chaînes de caractères qui désignent un même objet (une même étoile ou une même constellation). Un objet *ObjetCeleste* possède au moins une désignation qui doit être fournie au constructeur de la classe qui aura ce seul paramètre. La méthode *ajouter_une_designation()* permet d'ajouter une nouvelle désignation. Cette méthode déclenche aussi une exception en cas d'ajout d'une désignation qui existe déjà. La méthode *get_designations()* retourne la liste des désignations.

2.1 - Définir la classe *ObjetCeleste* en faisant respecter toutes les spécifications indiquées ci-dessus.

Un objet de la classe *Etoile* est en plus caractérisé par un attribut de type *float* qui représente la magnitude de l'étoile (une mesure de son irradiance). Le *getter* et le *setter* permettront de définir et de connaître sa valeur, le *setter* vérifiera que sa valeur est de type réel. Le constructeur de la classe prend deux paramètres qui représentent une première désignation pour l'étoile à créer et sa magnitude.

2.2 - Définir la classe *Etoile* en faisant respecter toutes les spécifications indiquées ci-dessus.

Un objet *Constellation* est en plus caractérisé par un attribut qui stocke les objets *Etoile* impliqués dans la constellation. Le constructeur de la classe *Constellation* a un paramètre représentant une première désignation de la constellation et deux paramètres de type *Etoile* qui représentent deux des étoiles impliquées dans la constellation. La méthode *ajouter_une_etoile()* prend en paramètre un objet *Etoile* permet d'ajouter une étoile à la constellation. Si on tente d'ajouter une étoile déjà existante, une exception doit être déclenchée.

2.3 - Définir la classe *Constellation* en faisant respecter toutes les spécifications indiquées ci-dessus.

2.4 - Modifier la classe *ObjetCeleste* de manière à enrichir au fur et à mesure de l'ajout des descriptions un index représenté par un attribut de classe. Cet index est un dictionnaire faisant correspondre à chaque clé représentant une désignation, l'objet céleste correspondant. À chaque fois qu'une désignation est ajoutée à un objet céleste, l'index est enrichi. Écrire la méthode de classe *get_objet_celeste()* qui prend en paramètre une désignation et renvoie l'objet céleste correspondant.

Cette méthode renvoie *None* si aucun objet céleste avec cette désignation existe.

2.5 – Écrire un programme principal quiinstanciera deux étoiles,instanciera une constellation avec ces deux étoiles, puis ajoutera une désignation à la constellation. Le programme demandera ensuite à l'utilisateur une désignation d'objet céleste et affichera par l'intermédiaire de la méthode *get_objet_celeste()* l'objet céleste s'il existe.

Exemple de corrigé :

```
class ObjetCeleste:
    index = {}
    def __init__(self, designation):
        self.__designation = []
        self.ajoute_une_designation(designation)

    def ajoute_une_designation(self, designation):
        if designation in self.__designation:
            raise TypeError("Designation existante")
        self.__designation.append(designation)
        ObjetCeleste.index[designation] = self

    def get_designation(self):
        return self.__designation

    def print(self):
        print(f"Désignation de l'objet celeste : ")
        for i in self.__designation:
            print(i)

    def get_objet_celeste(designation):
        if designation in ObjetCeleste.index.keys():
            return ObjetCeleste.index[designation]
        else:
            return None

class Etoile(ObjetCeleste):
    def __init__(self, designation, magnitude):
        super().__init__(designation)
        self.magnitude = magnitude

    @property
    def magnitude(self):
        return self.__magnitude

    @magnitude.setter
    def magnitude(self, val_magnitude):
        if not isinstance(val_magnitude, float):
            raise TypeError("magnitude doit être un réel")
        self.__magnitude = val_magnitude

    def print(self):
        print("Etoile : ")
        ObjetCeleste.print(self)
        print(f"Magnitude : {self.magnitude}")

    def __str__(self):
        return "Etoile"

class Constellation(ObjetCeleste):
    def __init__(self, designation, etoile1, etoile2):
        super().__init__(designation)
        self.__etoiles = []
        self.ajoute_une_etoile(etoile1)
        self.ajoute_une_etoile(etoile2)

    def ajoute_une_etoile(self, etoile):
        for i in self.__etoiles:
            if i == etoile:
                raise TypeError("Etoile existante")
        self.__etoiles.append(etoile)

    def print(self):
        print("Constellation")
        ObjetCeleste.print(self)
        for i in self.__etoiles:
            i.print()
```

```
def __str__(self):
    return "Constellation"

def main():

    etoile1 = Etoile("Aldébaran", magnitude = 12.)
    etoile1.ajoute_une_designation("Alpha-Tau")
    etoile1.ajoute_une_designation("HR 1457")

    etoile2 = Etoile("Mars", magnitude = 18.)
    etoile3 = Etoile("Jupiter", magnitude = 19.)
    etoile4 = Etoile("Venus", magnitude = 24.)

    # etoile1.print()

    grande_ours = Constellation("Grande ours",etoile4,etoile3)
    #
    grande_ours.ajoute_une_etoile(etoile1)
    #
    # grande_ours.print()
    #
    print(ObjetCeleste.get_objet_celeste("Alpha-Tau"))
    print(ObjetCeleste.get_objet_celeste("Grande ours"))

if __name__ == '__main__':
    main()
```